exness_bot — Step-by-Step Guide

LLM + rule-based auto-trading bot for Exness (MetaTrader 5)

Version 1.2 · Author: Waleed Hassan · Runs on Windows 7, 10 and 11

> **Read this first.** Automated FX/CFD trading loses money for most retail traders. **No bot can guarantee profit or "no losses" — this one cannot either.** The conservative defaults (small size, 2%/day loss cap, 10% total-loss kill switch, DRY-RUN, demo lock) keep a bad run *small*; they do not make it profitable. This bot is a **framework and starting point** and has **not** been run against a live account by the author. Always go **backtest → demo for weeks → tiny live size**, in that order. Algo trading is allowed by Exness; arbitrage, latency exploitation and quote-delay tick-scalping are not. Nothing here is financial advice.

Requirements (full list)

To backtest only (any OS, no broker needed)

Need	Detail
Operating system	Windows, macOS or Linux
Python	3.8–3.12, **64-bit** (tick *Add Python to PATH*)
Python packages	`pandas`, `numpy` — `pip install pandas` pulls both
Data	one CSV of candles, columns `time,open,high,low,close`

To trade (demo or live)

Need	Detail
Operating system	**Windows 7 SP1 / 10 / 11, 64-bit.** The `MetaTrader5` package is Windows-only; a Windows VPS also counts.
Python	**Windows 7 → Python 3.8.10** (last version that installs on Win 7). **Windows 10 / 11 → latest Python 3.** 64-bit, *Add Python to PATH*.
Python packages	`MetaTrader5>=5.0.37`, `pandas>=1.3.5`, `numpy>=1.21`; plus `openai>=1.10,<2` only for LLM mode. All installed by `pip install -r requirements.txt`.
Broker terminal	**Exness MetaTrader 5** desktop terminal, installed and logged in.
Exness account	Start with a **demo** account — you need its *login number*, *password* and *server* (e.g. `Exness-MT5Trial`).
Terminal setting	**Tools → Options → Expert Advisors → Allow algorithmic trading** ticked.
Network / uptime	Stable internet; the PC or VPS stays on and the terminal stays open while the bot runs.
Disk / RAM	~1 GB free disk (Python + MT5), 2 GB+ RAM.
Optional	An **OpenAI API key** — only if `USE_LLM = True` and you want LLM decisions instead of the built-in rules.

Hardware needs are light: the bot polls every few seconds and holds one position at a time. Any machine that runs the MT5 terminal comfortably runs the bot.

Part A — The easy way (no coding needed)

If you are not a programmer, follow just this part. You need a **Windows PC** (7, 10 or 11), a **free Exness demo account**, and about **20 minutes**. You will not type any code — only double-click a few files and fill in one form.

1. **Step 1 — Install Python (one time)**

   Go to `python.org/downloads`.
   - **Windows 7:** download **Python 3.8.10** (page: `python.org/downloads/release/python-3810/` → "Windows installer 64-bit"). Newer Python versions do *not* install on Windows 7.
   - **Windows 10 / 11:** download the latest Python 3.

   Run the installer.
   Tick the box "Add Python to PATH"
   , then click *Install Now*.

2. **Step 2 — Download the bot**

   Open `github.com/waleed260/exness-bot`. Click the green
   Code
   button →
   Download ZIP
   . Right-click the downloaded ZIP → *Extract All*. Remember where the folder is (e.g. your Desktop).

3. **Step 3 — Install MetaTrader 5 from Exness**

   In your Exness Personal Area, create a
   demo
   account and note its *login number*, *password* and *server* (e.g. `Exness-MT5Trial`). Download and install the
   Exness MetaTrader 5
   terminal and log into that demo account. In MT5:
   Tools → Options → Expert Advisors
   → tick
   "Allow algorithmic trading"
   → OK.

4. **Step 4 — Set it up (one time)**

   Open the extracted bot folder. Double-click
   `install.bat`
   . A black window installs everything (a few minutes), then a small text file opens in Notepad. Fill in your demo *login number*, *password* and *server* between the quote marks, then
   Save

(`Ctrl`+`S`) and close Notepad.

If it says "Python is not installed", you missed the "Add Python to PATH" tick in Step 1 — re-run the Python installer and choose *Modify*.

5. **Step 5 — Run it**

   Double-click `run.bat`. A black window opens and starts printing what the bot is doing. It is in **safe DRY-RUN mode**: it makes decisions and writes them down but **does not place any real orders**. Leave it running as long as you like.

   To stop: click the window and press `Ctrl`+`C`, or just close the window.

6. **Step 6 — See what it did**

   Inside the bot folder open `exness_bot\logs\`: `bot.log` is the full running commentary; `trades.csv` lists every trade it opened/closed (open it in Excel).

7. **Step 7 — Test on past data (optional but recommended)**

   In MetaTrader 5: *View → Symbols → Bars*, pick your pair and timeframe, *Export Bars* to a CSV file. Then double-click `backtest.bat` and drag that CSV into the window. It prints whether the strategy would have made or lost money, and a plain **EDGE / NO EDGE** verdict.

8. **Step 8 — Only when you are ready for more**

   - *Automatic demo orders* (still fake money): open `exness_bot\config.py` and set `EXECUTION_MODE = "demo"`, `DRY_RUN = False`, while keeping `DEMO_ONLY = True`. Confirm the startup log says `mode=demo` before leaving it running.
   - *Real money:* live automatic trading is disabled in this project. Do not change the live gate as part of normal setup.

> **Golden rules.** 1) Never skip the demo stage. 2) Never use money you cannot afford to lose. 3) The built-in strategy is a *starting point*, not a money machine — prove it with the backtester and the demo first.

The rest of this document (Parts 1–12) is the detailed reference: what each part does, every setting, command-line usage, an honest assessment for Exness, and troubleshooting.

1. What the bot does

> **Safe development status.** The default execution mode is `simulation` with `DRY_RUN=True`. The local simulator can open, monitor, apply SL/TP, close, persist, and recover test positions without an MT5 account. Live automatic trading is disabled by `LIVE_TRADING_ENABLED=False`. Creating `exness_bot/STOP_TRADING` blocks all new entries.

How to use it in simple stages

1. **Backtest** the strategy on historical candles.
2. **Run local simulation** with `python -m exness_bot.simulate --csv ...`. It makes decisions and manages simulated positions without MT5.
3. **Run demo dry-run** with `DRY_RUN=True`. It reads demo prices and logs decisions but sends no orders.
4. **Run automatic demo execution** only after reviewing the dry-run: set `EXECUTION_MODE = "demo"`, `DRY_RUN = False`, and keep `DEMO_ONLY = True`.

Automatic trading means the program receives data, waits for closed candles, calculates indicators, chooses `BUY`, `SELL`, or `NO-TRADE`, performs safety checks, and manages approved demo or simulated positions without a human clicking each action. It does not guarantee profit.

Press `Ctrl+C` to stop normally. Create `exness_bot/STOP_TRADING` to block new entries immediately; it does not forcibly close existing positions.

Once per **closed candle** (default timeframe M15) it runs this pipeline:

```
MT5 price history
      ▼
indicators   SMA(20), SMA(50), RSI(14), ATR(14), 200-SMA trend
      ▼
strategy     LLM decision (OpenAI)  --falls back to-->  rule set
      ▼                                    (SMA crossover + RSI + trend)
decision     buy / sell / close / hold   (+ confidence 0..1)
      ▼
filters      session window? spread ok? min ATR? confidence >= min?
             daily loss limit not hit? no position already open?
      ▼
risk         lot size from RISK_PER_TRADE_PCT
             SL = SL_ATR_MULT x ATR,  TP = TP_ATR_MULT x ATR
      ▼
executor     mt5.order_send (retries on requote)   [skipped if DRY_RUN]
```

On **every poll** (default every 5 s) it also manages any open position:

- at **+1.0R** profit → move the stop loss to entry (break-even);
- at **+1.5R** profit → trail the stop `2 × ATR` behind price;
- the stop is only ever tightened, never loosened.

`R` = the initial risk distance (entry → stop loss). All activity is logged to `exness_bot/logs/bot.log`; each entry/exit is appended to `exness_bot/logs/trades.csv`.

2. What every file is for

File	Job
`exness_bot/config.py`	all settings. `DRY_RUN=True`, `DEMO_ONLY=True` by default
`exness_bot/settings.example.py`	copy to `settings.py`; MT5 login + optional OpenAI key (git-ignored)
`exness_bot/mt5_client.py`	connect, account, positions, candles; auto-resolves the broker symbol suffix; reports spread
`exness_bot/indicators.py`	indicator maths + trailing-stop + session logic. No MT5 import, so the backtester reuses it
`exness_bot/data.py`	pulls live candles → DataFrame with indicator columns; drops the still-forming candle
`exness_bot/strategy.py`	picks a prompt, applies the cost guardrails, calls the LLM, validates the JSON; rule-based fallback
`exness_bot/prompts.py`	five named LLM prompt presets; `config.LLM_PROMPT_NAME` chooses one
`exness_bot/llm_guard.py`	keeps OpenAI spend low: only-on-signal, per-day cap, daily $ limit, min gap, snapshot cache, spend estimate
`exness_bot/risk.py`	lot sizing, SL/TP, break-even + trailing stop, daily-loss guard, session check
`exness_bot/executor.py`	`mt5.order_send` for open / close / modify-stop, with retries; obeys `DRY_RUN`
`exness_bot/runner.py`	the main loop that ties it together
`exness_bot/backtest.py`	runs the rule strategy over history and prints edge metrics
`install.bat` / `run.bat` / `backtest.bat`	Windows double-click helpers (Part A) — they just call the commands below for you

3. Every setting in `config.py`

Safety

Setting	Default	Meaning
`DRY_RUN`	`True`	`True` = never send a real order, only log what it would do
`DEMO_ONLY`	`True`	refuse to start if the connected account is a live account
`CONFIRM_LIVE_STRING`	`""`	must equal `"I ACCEPT THE RISK"` to run on a live account
`EXECUTION_MODE`	`"simulation"`	`simulation`, `demo`, or `live`; simulation is the default
`LIVE_TRADING_ENABLED`	`False`	hard gate; remains disabled during development
`EMERGENCY_STOP_FILE`	`exness_bot/STOP_TRADING`	presence blocks new entries
`MAX_TRADES_PER_DAY`	`10`	daily entry limit
`MAX_CONSECUTIVE_LOSSES`	`3`	blocks new entries after consecutive losses
`MAX_DATA_AGE_SECONDS`	`7200`	rejects stale candles before analysis

Market

Setting	Default	Meaning
`SYMBOL`	`EURUSD`	base name; broker suffix (`m` , `.` , `z` …) found automatically
`TIMEFRAME`	`M15`	candle used for decisions
`LOOKBACK_BARS`	`400`	candles pulled for indicators / LLM context

Risk

Defaults are deliberately conservative — they cap the damage of a bad run, they do **not** guarantee a profit.

Setting	Default	Meaning
`RISK_PER_TRADE_PCT`	`0.25`	percent of balance lost if the stop is hit
`FIXED_LOT_FALLBACK`	`0.01`	lot used if sizing can't be computed (also clamped to `MAX_LOT`)
`MAX_LOT`	`0.05`	hard cap on lot size
`MAX_OPEN_POSITIONS`	`1`	positions allowed at once on this symbol
`ATR_PERIOD`	`14`	ATR length
`SL_ATR_MULT`	`2.0`	stop distance = this × ATR
`TP_ATR_MULT`	`3.0`	target distance = this × ATR
`MIN_ATR_POINTS`	`0`	skip entries when ATR (points) is below this; `0` = off
`DAILY_MAX_LOSS_PCT`	`2.0`	stop opening trades for the day after this equity drawdown
`MAX_TOTAL_LOSS_PCT`	`10.0`	**hard kill switch** — stop the bot for good if equity falls this far below the equity it launched with; `0` = off
`MIN_CONFIDENCE`	`0.55`	ignore buy/sell signals weaker than this

Trailing / break-even

Setting	Default	Meaning
`BREAKEVEN_AT_R`	`1.0`	move stop to entry once profit reaches this many R; `0` = off
`TRAIL_AT_R`	`1.5`	start trailing once profit reaches this many R; `0` = off
`TRAIL_ATR_MULT`	`2.0`	trailing stop sits this × ATR behind price

Entry filters

Setting	Default	Meaning
`MAX_SPREAD_POINTS`	`25`	skip entries when the spread is wider than this
`USE_TREND_FILTER`	`True`	only long in an up-trend, only short in a down-trend
`TREND_SMA`	`200`	SMA used as the trend proxy
`TREND_SLOPE_LOOKBACK`	`20`	bars back used to measure the trend SMA slope
`SESSION_UTC_HOURS`	`[7, 16]`	trade only between these UTC hours; `[]` = always
`TRADE_DAYS`	`[0,1,2,3,4]`	weekdays allowed (Mon–Fri)

Execution

Setting	Default	Meaning
`DEVIATION_POINTS`	`20`	maximum slippage accepted, in points
`MAGIC`	`20240601`	id stamped on this bot's orders
`POLL_SECONDS`	`5`	how often the loop checks for a new candle / manages the stop
`ORDER_RETRIES`	`3`	retries on requote / price-changed
`TRADE_LOG_CSV`	`exness_bot/logs/trades.csv`	where trades are recorded

Strategy

Setting	Default	Meaning
`USE_LLM`	`True`	use the LLM if an API key is set, else the rule set
`SMA_FAST` / `SMA_SLOW`	`20` / `50`	crossover SMAs
`RSI_PERIOD`	`14`	RSI length

LLM cost guardrails

Only apply when `USE_LLM=True` **and** `settings.OPENAI_API_KEY` is set. Full walkthrough in Part 7.

Setting	Default	Meaning
`LLM_PROMPT_NAME`	`"conservative"`	preset from `prompts.py`: `conservative` / `trend_follow` / `mean_reversion` / `breakout` / `swing`
`LLM_ONLY_ON_SIGNAL`	`True`	call the model **only** when the rule engine already sees a buy/sell/close setup — the biggest cost saver
`LLM_ENTRIES_ONLY`	`True`	never spend a call on an exit — rules + SL/TP + trailing stop close positions; only entries go to the model
`LLM_MIN_SECONDS_BETWEEN_CALLS`	`300`	hard floor on call frequency, whatever the timeframe
`LLM_MAX_CALLS_PER_DAY`	`20`	hard cap per UTC day; `0` = unlimited
`LLM_DAILY_COST_LIMIT_USD`	`0.15`	stop calling once the day's *estimated* spend hits this; `0` = off
`LLM_SKIP_OUTSIDE_SESSION`	`True`	no calls outside `SESSION_UTC_HOURS` / `TRADE_DAYS`
`LLM_CACHE_SNAPSHOT`	`True`	reuse the last decision when the snapshot barely moved (no call)
`LLM_SEND_PRICE_HISTORY`	`False`	include `last_10_closes` in the prompt (more tokens; rarely worth it)
`LLM_MAX_OUTPUT_TOKENS`	`40`	reply is a tiny JSON object; capped low
`LLM_COST_PER_1K_INPUT` / `_OUTPUT`	`0.00015` / `0.0006`	fallback price for the log estimate; a known `OPENAI_MODEL` uses its own price automatically

Backtest

Setting	Default	Meaning
`BT_SPREAD_POINTS`	`12`	spread cost (points) charged on every backtested trade
`BT_COMMISSION_PER_LOT`	`7.0`	informational commission assumption for Raw/Zero accounts
`BT_START_BALANCE`	`1000`	starting balance for the money curve

4. Step-by-step: backtest (do this FIRST)

Why first: the backtest tells you whether the strategy has any edge on past data *before* you risk a cent. If it says NO EDGE, do not run it on demo or live — change the settings and test again. Runs on **any OS** (Windows / macOS / Linux) — no MetaTrader, no broker needed.

Step 1 — install Python + pandas

In the VS Code terminal (or any terminal), from the `exness-bot` folder:

```
pip install pandas
```

That also pulls `numpy`. You do **not** need `MetaTrader5` for a CSV backtest.

Step 2 — get historical candles as a CSV

One CSV with columns `time, open, high, low, close` (extra columns are ignored; headers like `<OPEN>` / `Date` are auto-mapped).

From the Exness MT5 terminal (best):

1. MT5 → **View** → **Symbols** (or `Ctrl`+`U`).
2. Pick your pair (e.g. `EURUSD`) → tab **Bars**.
3. **Timeframe = M15** (match `config.TIMEFRAME`), pick a wide date range (2+ years), click **Request** then **Export Bars** → save as `EURUSD_M15.csv`.
4. Make a `data` folder inside `exness-bot`, move the file to `data/EURUSD_M15.csv`.

Or a free source: any OHLC-candle provider (Dukascopy, HistData, TraderMade…). Save it with those column names.

Step 3 — run the backtest

```
python -m exness_bot.backtest --csv data/EURUSD_M15.csv
```

On Windows with the MT5 terminal open and `settings.py` filled in, you can skip the CSV and pull data straight from the broker:

```
python -m exness_bot.backtest --mt5 180        # last 180 days of config.TIMEFRAME
```

Step 4 — read the report

```
data span      : 2023-01-02 -> 2025-06-30 (910 days)
trades         : 148
win rate       : 41.9%
avg win / loss : +1.85R / -1.00R
expectancy     : +0.043R per trade
profit factor  : 1.21
total          : +6.4R
max drawdown   : 7.8%
balance        : 1000 -> 1187  (+19%)
verdict        : EDGE? maybe - forward test on demo
```

Result	Meaning
PASS	`expectancy > 0` *and* `profit factor > 1.15` *and* a believable trade count (say ≥ 100) → go to Part 5, forward-test on **demo**
FAIL	anything else, or `verdict : NO EDGE` → do **not** trade it; go to Step 5

Step 5 — if it fails, tune and re-run

Change **one thing at a time** in `exness_bot/config.py`, save, run Step 3 again, compare:

- `TIMEFRAME` (`_M5` / `_M15` / `_H1` …)
- `SMA_FAST` / `SMA_SLOW`, `RSI_PERIOD`
- `SL_ATR_MULT` / `TP_ATR_MULT`
- filters: `USE_TREND_FILTER`, `SESSION_UTC_HOURS`, `MIN_CONFIDENCE`

Per-trade detail for the last run is written to `exness_bot/logs/backtest_trades.csv` (open in Excel).

Step 6 — validate before trusting it

Even a PASS can be luck. Before demo:

- Re-run on a **different pair** (e.g. `GBPUSD`) — still positive?
- Tune on the first half of the data, then run **untouched** on the second half — still positive? (walk-forward)
- Only then move to Part 5.

> **Never** move to demo-real or live money on a “NO EDGE” result. The built-in SMA/RSI strategy is a starting point and usually shows no edge as-is.

5. Step-by-step: demo on Windows

The `MetaTrader5` Python package only runs on **Windows** (a Windows VPS is fine).

Python version: on **Windows 7** use **Python 3.8.10** — the last release that installs on Win 7, and `requirements.txt` is set so the right package versions install automatically. On **Windows 10 / 11** use the latest Python 3. Either way tick *"Add Python to PATH"* and use the **64-bit** installer.

1. Create a **demo** account in the Exness Personal Area; note the login number, password and server (e.g. `Exness-MT5Trial`).
2. Install the **Exness MetaTrader 5** terminal, log into the demo account.
3. In the terminal: **Tools → Options → Expert Advisors → Allow algorithmic trading** (tick it).
4. Install dependencies:

   ```
   pip install -r requirements.txt
   ```

5. Create your private config:

   ```
   copy exness_bot\settings.example.py exness_bot\settings.py
   ```

   Edit `exness_bot\settings.py`:

   ```
   MT5_PATH     = r"C:\Program Files\MetaTrader 5 EXNESS\terminal64.exe"
   MT5_LOGIN    = 12345678
   MT5_PASSWORD = "your-demo-password"
   MT5_SERVER   = "Exness-MT5Trial"
   OPENAI_API_KEY = ""          # leave empty to use the rule-based strategy
   OPENAI_MODEL   = "gpt-4o-mini"
   ```

   (`settings.py` is git-ignored — credentials never get committed.)
6. First run in safe simulation/dry-run mode. Keep `EXECUTION_MODE = "simulation"` and `DRY_RUN = True`:

   ```
   python -m exness_bot.runner
   ```

   Watch `exness_bot/logs/bot.log`. No external orders are sent.
7. After the dry-run looks sane, set `EXECUTION_MODE = "demo"` and `DRY_RUN = False`. Keep `DEMO_ONLY = True`, confirm the MT5 terminal is logged into the demo account, and check the startup log says `mode=demo`. This sends automatic orders to the demo account only. Let it run for **several weeks**; review `logs/trades.csv`.
8. Stop with `Ctrl+C`. For an emergency stop, create `exness_bot\STOP_TRADING`; this blocks new entries.

6. Live preparation (disabled)

> **Live automatic trading is disabled.** Do not change `LIVE_TRADING_ENABLED`, `DEMO_ONLY`, or the live confirmation gate as part of normal use. A future live release would require a separate review, fresh tests, and explicit authorization.

7. LLM mode & keeping the OpenAI bill tiny

The bot runs fully **without** OpenAI — the rule engine is the default and costs **$0**. LLM mode adds a second opinion: on a *potential* trade it asks a model for a `buy / sell / close / hold` JSON decision, and still falls back to the rule result if anything goes wrong.

7.1 Add your API key (do it in VS Code)

1. **Create a key** at `platform.openai.com/api-keys`.
2. **Set a hard monthly spend limit and turn auto-recharge OFF** at `platform.openai.com/settings/organization/limits` (e.g. $5). This is the ceiling that actually protects you.
3. In VS Code open **`exness_bot/settings.py`** (the file you made from `settings.example.py`) and paste the key:

   ```
   OPENAI_API_KEY = "sk-...your key..."
   OPENAI_MODEL   = "gpt-4o-mini"
   ```

   Save. `settings.py` is git-ignored, so the key is never committed.
4. In `exness_bot/config.py` keep `USE_LLM = True`. The guardrails below are already switched on.

7.2 Which model handles trades best?

`gpt-4o-mini` is the default and is fine for this simple SMA/RSI strategy. For stronger judgement on messy / borderline setups, set a bigger model in `settings.py` — the guardrails keep even the big ones at cents per month.

OPENAI_MODEL	When to use it	~cost / decision
`gpt-4o-mini` (default)	simple strategy, tightest budget	~$0.00006
`gpt-4.1-mini`	a bit sharper, still very cheap	~$0.0002
`gpt-4o`	best at reading unclear / conflicting setups	~$0.001
`gpt-4.1`	strongest reasoning of the four	~$0.002

`llm_guard.py` knows these prices, so the spend line in the log stays accurate whichever you pick; a newer model id just falls back to the `LLM_COST_PER_1K_*` estimate.

7.3 What keeps the cost down

Every guardrail lives in the **LLM cost guardrails** block of `config.py` (table in Part 3). In plain terms:

- **Only on a signal.** `LLM_ONLY_ON_SIGNAL=True` — the model is called *only* when the free rule engine already sees a setup. Quiet candles cost nothing. This alone removes most calls.
- **Entries only.** `LLM_ENTRIES_ONLY=True` — the model is never asked about closing a trade; rules, the broker SL/TP and the trailing stop handle exits. Roughly halves what is left.

- **A hard daily budget.** `LLM_MAX_CALLS_PER_DAY=20` and `LLM_DAILY_COST_LIMIT_USD=0.15` — whichever is hit first, the bot goes rules-only for the rest of the UTC day.
- **A minimum gap.** `LLM_MIN_SECONDS_BETWEEN_CALLS=300` — at most one call per 5 minutes.
- **Session-aware.** `LLM_SKIP_OUTSIDE_SESSION=True` — no calls at night or on weekends.
- **Caching.** `LLM_CACHE_SNAPSHOT=True` — if the market barely moved, the previous answer is reused with no new call.
- **Small prompts.** A compact one-line snapshot, `last_10_closes` left out, reply capped at `LLM_MAX_OUTPUT_TOKENS=40`, and `response_format=json_object` so a reply is never wasted.

`logs/bot.log` prints a live estimate on every call:

```
LLM call 3/20 today | ~256 in / 20 out tok | est $0.00005 | est day total $0.0002
```

7.4 Rough cost

With `gpt-4o-mini`, one decision is about **$0.00006** (~260 input + 20 output tokens). Even flat-out at 20 calls/day that is **a few cents a month**; with *only-on-signal* + *entries-only* it is usually a handful of calls a day. With `gpt-4o` multiply by ~15 and it is still well under a dollar a month. Your dashboard limit from step 2 is the real cap.

7.5 Choosing a prompt

`exness_bot/prompts.py` ships five presets. Set the one you want in `config.py`: `LLM_PROMPT_NAME = "conservative"`.

Name	Style
`conservative` (default)	trend-aligned, textbook setups only, holds when unsure
`trend_follow`	rides the 200-SMA trend, lets winners run
`mean_reversion`	fades RSI extremes back toward the slow SMA
`breakout`	momentum breakouts on expanding ATR, in the trend direction
`swing`	rare, high-confluence entries; expects long holds

To write your own: add another template to `PRESETS` in `prompts.py` (keep `+ _CONTRACT` on the end so the JSON reply still parses), then point `LLM_PROMPT_NAME` at it.

8. Run it in VS Code (for developers)

1. **Install VS Code**
   and the
   Python extension
   (Microsoft).
2. **File → Open Folder…**
   and pick the extracted `exness-bot` folder.
3. **Terminal → New Terminal**
   to get a shell at the project root.
4. *(Recommended)* create a virtual environment so packages stay local:

   ```
   python -m venv .venv
   .venv\Scripts\activate          # Windows
   # source .venv/bin/activate     # macOS / Linux (backtest only)
   ```

   Then click the Python version in the status bar (bottom-right) and select the `.venv` interpreter.
5. Install packages:

   ```
   pip install -r requirements.txt      # trading  (Windows)
   pip install pandas                   # backtest only (any OS)
   ```

6. Create the settings file once:

   ```
   copy exness_bot\settings.example.py exness_bot\settings.py    # Windows
   ```

   Open `exness_bot/settings.py` and fill in the demo login / password / server. For LLM mode, paste `OPENAI_API_KEY` here too — see
   Part 7
   . Cost knobs and the prompt preset live in `exness_bot/config.py` (`LLM cost guardrails`).
7. Run from the VS Code terminal:
   - Backtest — `python -m exness_bot.backtest --csv data/EURUSD_M15.csv`
   - Bot (safe DRY-RUN) — `python -m exness_bot.runner`
   - Or open `exness_bot/runner.py` and press `F5` to run under the debugger (set breakpoints in `strategy.py` / `risk.py` to watch a decision form).
8. Stop the bot with `Ctrl`+`C` in the terminal.

Output goes to `exness_bot/logs/bot.log` and `exness_bot/logs/trades.csv` — open them in the editor while it runs.

9. Deploy — where to run it and how (easy way)

What "deploy" means here: the bot has to stay running, and the **Exness MT5 terminal has to stay open and logged in** next to it. There is no website to upload to. You just pick a Windows machine that is always on and start the bot on it.

Which machine?

Choice	Good for	Notes
Your own PC	trying it, demo weeks	free; the PC must stay on 24/5 and not sleep
A Windows VPS	real / unattended use	~$10–30/mo; always on; search "Windows VPS" or "Forex VPS"

macOS / Linux / phones / free cloud containers **cannot** run it — the `MetaTrader5` package needs a real Windows session. (Linux is fine for backtesting only.)

Easy way — your own PC

1. Finish Part A (or Part 8) so `run.bat` already works once.
2. *Settings* → *Power & sleep* → set **Sleep = Never** (turning the screen off is fine; sleep is not).
3. Open the **Exness MT5** terminal, log in, leave it open.
4. Double-click **`run.bat`**. Leave the black window open. Done.
5. Check `exness_bot\logs\bot.log` now and then.

Easy way — a Windows VPS (recommended once past demo)

1. Buy a small **Windows Server VPS** (2 vCPU / 4 GB RAM is plenty).
2. On your PC press `Win`+`R`, type `mstsc`, enter the VPS address / user / password to get its desktop.
3. On the VPS do the normal one-time setup: install **Python** (same version rule as Part 5), install the **Exness MT5** terminal and log in, tick *Allow algorithmic trading*.
4. Copy the `exness-bot` folder onto the VPS (drag-and-drop into the Remote Desktop window). Run **`install.bat`**, fill in `settings.py`.
5. Run **`run.bat`**. Now close the Remote Desktop window (do **not** "Sign out") — the bot and MT5 keep running on the VPS.
6. Reconnect with `mstsc` any time to check on it.

Make it restart itself (optional, for real use)

So it comes back after a crash or a VPS reboot:

- **MT5:** add the terminal to Windows startup so it launches on boot.
- **The bot:** install **NSSM** (free), then in a terminal:

```
nssm install exness-bot "C:\path\to\python.exe" -m exness_bot.runner
nssm set exness-bot AppDirectory "C:\path\to\exness-bot"
nssm start exness-bot
```

Windows now keeps the bot running and restarts it if it exits. Or use **Task Scheduler** → new task → *Run whether user is logged on or not* + *Restart on failure*.

Check `logs/bot.log` every day for the first week.

No cloud / Docker option. `MetaTrader5` needs a real Windows session with the terminal running, so a Linux container cannot trade. A Linux box is fine for backtesting only.

10. How good is it for Exness — honest assessment

a) Will it work *technically* on Exness?

Fairly likely after a short debugging pass on demo. Usual first-run snags:

Snag	Fix
wrong `MT5_SERVER` string	`initialize failed` → copy the exact server name from the terminal
symbol is `EURUSDm` / `EURUSD.z` on your account	handled automatically; check the log line `Resolved symbol EURUSD -> ...`
"AutoTrading disabled"	step 5.3 above
broker min stop distance > your ATR stop	the bot clamps it; on very tight timeframes SL/TP get pushed out — use a higher timeframe

Rough chance of a clean demo run within a day or two: **~85%**.

b) Will it make real profit?

The built-in strategy (SMA crossover + RSI + trend filter, or a generic LLM prompt) is a **baseline, not an edge**. After spread, swap and commission its expected value is around zero-to-negative. Realistic odds that this exact configuration is net-profitable over a year: **~10–15%**, in line with retail algo-trading outcomes generally.

What actually moves that number:

- A strategy with a tested statistical edge — validate with `backtest.py` over several years and multiple symbols, then walk-forward.
- Trading the right sessions / instruments for that edge.
- Costs: a low-spread account type; avoid holding across the 3-day swap.
- Risk discipline: small `RISK_PER_TRADE_PCT`, a hard daily stop — which this bot already enforces.

> **Bottom line:** treat this as a solid *execution and risk* framework with a *backtester* attached. The profitable idea has to be yours and has to survive the backtest before any money is at stake.

11. Troubleshooting

Symptom	Cause / fix
`settings.py not found`	you didn't copy `settings.example.py` → `settings.py`
`mt5.initialize failed`	wrong `MT5_PATH`, `MT5_SERVER`, login or password; terminal not installed
`Symbol EURUSD(+suffix) not found`	your account uses a different name; add its suffix to `_SUFFIXES` in `mt5_client.py`
Decisions logged but no external orders	Expected in `simulation` mode or when `DRY_RUN` is `True`
`OPEN rejected: retcode=10027`	AutoTrading disabled in the terminal
`OPEN rejected: retcode=10016`	invalid stops (broker min distance); raise `SL_ATR_MULT` or use a higher timeframe
`OPEN rejected: retcode=10019`	not enough money for that lot; lower `RISK_PER_TRADE_PCT` or `MAX_LOT`
No trades ever	trend filter + session window too strict, or `MIN_CONFIDENCE` too high; loosen and re-backtest
LLM errors in the log	bad/empty `OPENAI_API_KEY`; the bot auto-falls back to the rule set
Log always says `Skipping LLM (...)`	that's the guardrails working — it calls the model only on a rule-side setup, within budget and session. Loosen the `LLM_*` settings in `config.py` for more calls (costs more).
`insufficient_quota` / `429` from OpenAI	add credit, or the dashboard spend limit is hit; the bot keeps trading rules-only meanwhile
Want $0 / no OpenAI at all	leave `OPENAI_API_KEY = ""` (or set `USE_LLM = False`) — pure rule engine
`OPEN rejected: retcode=10030` (Unsupported filling)	the bot now auto-retries FOK → IOC → RETURN; if it still fails, your account blocks market orders on that symbol
Bot exits with `MAX_TOTAL_LOSS_PCT hit`	equity fell 10% below launch — that's the kill switch. Review before restarting; adjust the % in `config.py`
Trade in the log but not in `trades.csv`	the order was rejected — a rejection is logged as an error and no longer written as a fake fill
`ModuleNotFoundError: MetaTrader5` on Win 10/11	you installed 32-bit Python, or Python is too new for a wheel — use the 64-bit installer and re-run `install.bat`
Python installer says "not supported on this OS" (Win 7)	you downloaded a version newer than 3.8.10 — get Python 3.8.10

12. Safe-launch checklist

[ ] Backtested on ≥ 2 years of data, on ≥ 2 symbols → positive expectancy, PF > 1.15

[ ] Walk-forward / out-of-sample check also positive

[ ] Ran on demo with `DRY_RUN=True`, decisions look sane in the log

[ ] Ran automatic demo mode with `EXECUTION_MODE="demo"` and `DRY_RUN=False` for ≥ 4 weeks → result matches the backtest

[ ] `RISK_PER_TRADE_PCT` ≤ 0.25, `DAILY_MAX_LOSS_PCT` ≤ 2, `MAX_LOT` small

[ ] `MAX_TOTAL_LOSS_PCT` set to a number you would actually accept losing

[ ] Account type has low spread; you understand swap on your instrument

[ ] VPS / machine stays on; MT5 terminal set to start with the OS

[ ] You can afford to lose the whole account balance

These checks prepare a future review; they do not enable live trading. The live flags remain disabled in this project.